

**CENTRO INTEGRADO DE FORMACIÓN PROFESIONAL DE AVILÉS
FAMILIA PROFESIONAL DE INFORMÁTICA Y COMUNICACIONES**

Ciclo Formativo de Grado Superior
Desarrollo de Aplicaciones Web (DAW) – 2º Curso

PROYECTO INTERMODULAR II
Desarrollo de Aplicaciones Web

Modalidad: Presencial
Tipo de proyecto: Desarrollo de aplicación web dinámica

Título del Proyecto: TopMakki
Subtítulo: Web de venta de camisetas de futbol

Documento-Memoria

Presentación del alumno	2
Título del proyecto y tipo de proyecto elegido	2
2.1 Descripción general y requisitos.....	3
3.1. Planificación de actividades y tareas. Temporización. Diagrama de Gantt.....	4
3.2. Estimación de costes.....	5
3.3. Previsión riesgos.....	6
4.1 Diseño y arquitectura de la aplicación	8
4.2 Modelo de datos y estructura	10
4.3 Modelo de procesos y servicios	12
5.1 Modelo de procesos y servicios	14
5.2 Modelo de procesos y servicios	16
5.3 Manual de usuario	19
.....	22
.....	23
5.4 Manual de instalación y despliegue.....	29
6.Evaluación final del proyecto.....	32
6.1 Evaluación del diseño y resultado final	33
Evolución técnica	34
Resultado final obtenido	34
Aspectos positivos del resultado.....	34
Aspectos mejorables	35
6.2 Conclusiones y lecciones aprendidas	35
7.Bibliografía y recursos	37
Documentación oficial	37
Tecnologías utilizadas	38
Recursos gráficos	38
Herramientas de desarrollo	38

1.Introducción

Presentación del alumno

Este proyecto de desarrollo de aplicaciones web ha sido realizado por Mohamed El Makki Belahsane, alumno del módulo Proyecto Intermodular II del Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web (DAW) en el CIFP Avilés, durante el curso académico 2025-2026 supervisado por Pablo Lera Menendez como tutor colectivo

Título del proyecto y tipo de proyecto elegido

El proyecto se titula TopMakki y nace del intento de intentar que los compradores busquen una alternativa más económica y llevadera frente a los precios abusivos de los grandes equipos por en un futbol más globalizado, permitiendo a los usuarios registrarse, escoger en una amplia gama de productos, filtrarlos por ligas, y personalizarlos mediante un sistema de carrito y checkout.

La aplicacion también cuenta con un usuario de administrador encargado de gestionar los productos de la tienda, que camisetas nuevas vender y cuales eliminar

Para su desarrollo se emplearán en el backend lenguajes como PHP y MySQL, y en frontend por otra parte se utilizará HTML5, CSS3 y JavaScript conectado por modelo vista controlador

2. Definición del proyecto

2.1 Descripción general y requisitos

La aplicación TopMakki nace de la idea de tener una tienda de ropa deportiva trasladada a los conocimientos adquiridos en este ciclo de Desarrollo de Aplicaciones Web, una web inspirada en otras plataformas deportivas, puesto que se está perdiendo un poco la tienda física frente la online y a la poca rentabilidad de alquilar locales y los gastos que ello conlleva, se ha decidido digitalizar el negocio y hacerlo más actualizado frente a la digitalización que está sufriendo nuestro mundo en estos últimos años. En esta App cumplimos con una serie de requisitos:

- **Inicio y registro obligatorio** antes de entrar a la aplicación, pues para poder comprar e incluso echar un vistazo, se deberá crear una cuenta para ello, y al cerrar sesión se volverá a la pantalla de inicio de sesión.
- **Gestión de productos**, de los cuales los usuarios podrán visualizar el amplio catálogo de camisetas, filtrar por ligas e incluso contar con un buscador dinámico para encontrar más fácilmente la camiseta en concreto.
- **El uso del carrito y de los pedidos**, en el que podrán los usuarios que así lo deseen añadir productos al carrito, personalizar camisetas, eliminar productos del carrito, finalizar y hasta pagar online por su adquisición.
- **La figura del administrador**, que cuenta con acceso restringido y solo el puede acceder a editar precios, eliminar y añadir camisetas para futuras ventas.

Requisitos técnicos

- Cumplimos con la seguridad de nuestros clientes cifrando contraseñas mediante password_hash, previniendo la inyección de código.
- La compatibilidad con otros navegadores también es uno de nuestros puntos a seguir, como Google Chrome, FireFox y Edge.
- El diseño responsive adaptable móvil, tablet y escritorio, accesible para todos los dispositivos
- Todo ello inspirado por la arquitectura Modelo Vista Controlador.

3. Planificación del proyecto

3.1. Planificación de actividades y tareas. Temporización. Diagrama de Gantt

Para el desarrollo final de “TopMakki”, se ha hecho uso de Diagrama de Gantt para que los requisitos anteriormente mencionados se traduzcan en una o varias actividades que serán llevadas a cabo y serán plasmadas en el anteriormente mencionado Diagrama de Gantt

Análisis del proyecto

- Definición de requisitos, en la que tenemos en cuenta los dos tipos de requisitos
- Diseño inicial.

Backend

- Conexión MySQL., mediante phpmyadmin, para albergar la base de datos, y un servidor, XAMPP, la conexión de la aplicación con la base de datos se hace realidad
- Sistema login.
- CRUD productos.
- Gestión carrito.

- Checkout.

Frontend

- Maquetación.
- Responsive.
- Formularios.

Seguridad

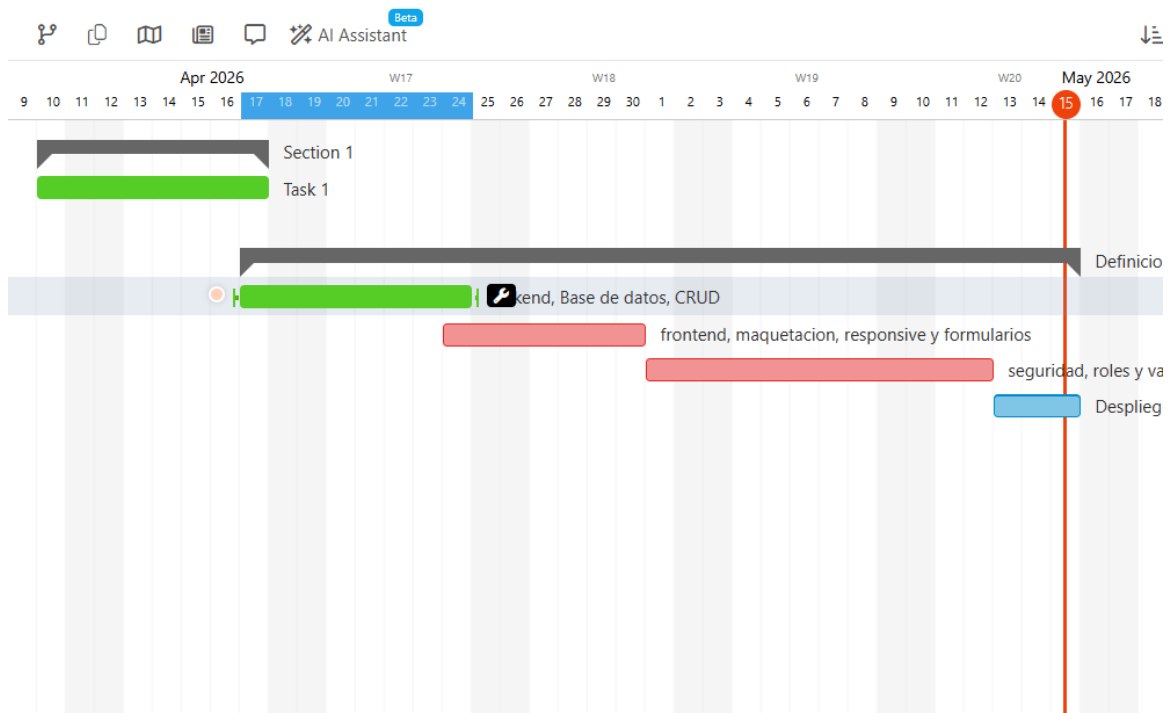
- Roles.
- Validaciones.
- Sesiones.

Testing

- Pruebas funcionales.
- Validación responsive.

Despliegue

- Preparación para hosting.



3.2. Estimación de costes

La estimación de costes se ha hecho realidad previsiblemente haciendo una estimación del desarrollo de una plataforma como TopMakki, los resultados son los siguientes:

Recursos humanos:

Perfil profesional	Horas estimadas	Coste/hora	Coste total
Desarrollador Full Stack	140 horas	18€/hora	2520€
Diseñador UI/UX	25 horas	16€/hora	400€
Testing documentación	20 horas	15€/hora	300€

Coste total recursos humanos: 3220€

Recursos materiales:

Recurso	Coste
Ordenador personal	900€
Visual Studio Code	Gratis
XAMPP	Gratis
phpMyAdmin	Gratis
Navegador de pruebas	Gratis

Coste total asciende a 4120€, teniendo en cuenta los perfiles de trabajo y las herramientas gratuitas ofrecidas. Al tratarse de un proyecto académico, gran parte de las herramientas empleadas han sido gratuitas, lo que ha reducido considerablemente los costes reales de implementación.

3.3. Previsión riesgos

En la realización del proyecto, nunca se puede dar por hecho siempre que no vayan a surgir contratiempos, y desgraciadamente en muchos casos el proyecto se puede ver afectado en ello

Riesgo	Probabilidad	Impacto	Medida preventiva / solución
Pérdida de datos de la base de datos	Media	Alta	Realización de copias de seguridad periódicas
Tiempo insuficiente para terminarlo	Media	Alta	Planificación y división por fases
Errores en el sistema login	Baja	Alta	Uso de pruebas funcionales y validaciones
Problemas responsive en móviles	Media	Media	Uso de media queries y pruebas multidispositivo
Problemas seguridad por SQL Injection	Baja	Alta	Uso prepared statements
Fallos en conexión con MySQL	Baja	Alta	Control de errores mediante PDO

La identificación previa de riesgos permitió anticiparse a posibles problemas durante el desarrollo del proyecto.

La mayoría de los riesgos pudieron minimizarse mediante:

- pruebas continuas,
- validación de datos,
- uso de sesiones,
- organización del código,
- y utilización de buenas prácticas de programación.

Esto ayudó a mejorar la estabilidad y seguridad de la aplicación final.

4. Análisis y diseño del programa

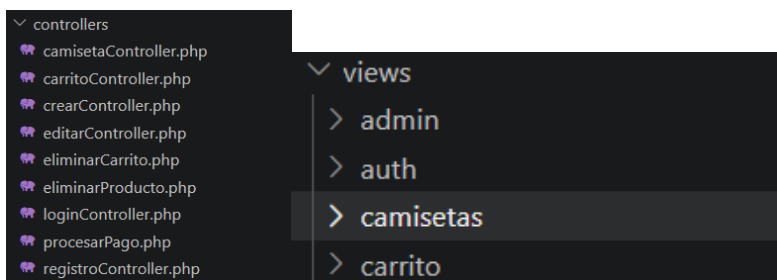
4.1 Diseño y arquitectura de la aplicación

En este apartado se describe la estructura general que ha mandado en la aplicación, las herramientas tecnológicas usadas y la organización interna del proyecto. El objetivo principal siempre ha sido que la aplicación estuviera organizada, mantenible, fácil de ampliar utilizando una jerarquía inspirada en el Modelo Vista Controlador(MVC), inspirada porque no contiene modelos, solo vistas y controladores, los modelos se iban a utilizar pero conforme avanzaba el proyecto no se hizo, se optó más por conexión directa PDO, separando parcialmente la lógica del negocio, las vistas y el acceso a datos.

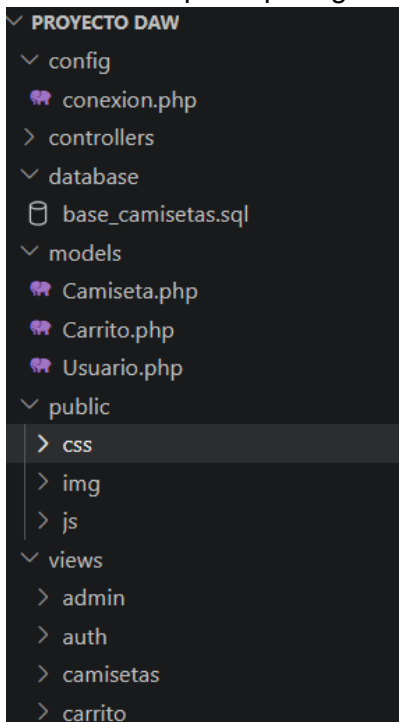
Este patrón nos permite separar el proyecto en 2 partes principales

- Vista: la capa vista es aquella que contiene todo lo que se va a visualizar en primera instancia, aquí nos vamos a encontrar que están desarrolladas con HTML5, que se ha utilizado como esqueleto del proyecto, estructurando la aplicación, CSS3, aportando contenido visual, estilos y maquetación de la aplicación, y JavaScript, que aunque no haya tenido tanta importancia como las 2 anteriores, aporta un buscador que permite buscar con más rapidez las camisetas deseadas, abajo se muestran las vistas también.
- Controlador: la capa controlador se encarga de conectar las vistas con los datos y gestionar la lógica del negocio, procesan, se encargan de procesar los formularios, las sesiones, validaciones, operaciones CRUD y acciones en el carrito. He aquí los

controladores utilizados en el proyecto:



- Estructura del proyecto: el proyecto como se puede intuir se encuentra organizado en diferentes carpetas para garantizar su funcionalidad:



- /config: la conexión a la base de datos
- /controllers:

loginController.php: se encarga del loggeo

registroController.php: administra los registros

carritoController.php: introduce las camisetas en los carritos

editarController.php: edita las camisetas por parte del admin

eliminarCarrito.php: elimina el carrito del comprador

eliminarProducto.php: admin elimina productos

loginController.php: procesa el pago de la mercancía

camisetaController.php: encargado de crear, eliminar y actualizar camisetas

procesarPago.php: calcula el total de lo que hay que pagar e inserta los pedidos en la base

Tecnologías utilizadas

- **Backend:** se ha escogido combinar tecnologías como PHP en la capa de negocio, aunque se dudó hacerlo con Laravel, finalmente se optó por PHP al tener más conocimiento sobre ello, y para como tecnología de almacenamiento se ha escogido MySQL y phpMyAdmin con tal de guardar carritos, camisetas y pedidos. Juntos, se encargan de la autenticación, las sesiones y el acceso a datos de la aplicación
- **Frontend:** como se ha mencionado anteriormente, se ha confiado en HTML5, CSS3 y JavaScript para favorecer la interacción con el usuario y mostrar la interfaz gráfica.
- **Herramientas de desarrollo:** el proyecto se ha desarrollado en VSCode como entorno de desarrollo, el servidor XAMPP ha sido crucial permitiendo la conexión de phpMyAdmin y PHP.

4.2 Modelo de datos y estructura

En este apartado se describe la estructura de la base de datos utilizada en la aplicación TopMakki, así como las relaciones entre las distintas tablas del sistema.

La base de datos ha sido diseñada con el objetivo de almacenar correctamente toda la información necesaria para el funcionamiento del ecommerce, incluyendo usuarios, productos, pedidos y detalles de compra.

El sistema utiliza MySQL como gestor de base de datos y la conexión se realiza mediante PHP PDO.

La base recibe el nombre de base_camisetas, y contiene las siguientes tablas

☐	camisetas	★							58	InnoDB	utf8mb4_general_ci	16.0 KB	-	
☐	detalle_pedido	★							7	InnoDB	utf8mb4_general_ci	16.0 KB	-	
☐	pedidos	★							3	InnoDB	utf8mb4_general_ci	16.0 KB	-	
☐	usuarios	★							4	InnoDB	utf8mb4_general_ci	16.0 KB	-	
4 tablas		Número de filas								72	InnoDB	utf8mb4_general_ci	64.0 KB	0 B

Estructura				tabla				usuarios:		
#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterminado	Comentarios	Extra		
☐ 1	id	int(11)			No	Ninguna		AUTO_INCREMENT		
☐ 2	nombre	varchar(100)	utf8mb4_general_ci		No	Ninguna				
☐ 3	email	varchar(100)	utf8mb4_general_ci		No	Ninguna				
☐ 4	password	varchar(255)	utf8mb4_general_ci		No	Ninguna				
☐ 5	rol	varchar(20)	utf8mb4_general_ci		Sí	comprador				

Funcionalidad

Esta tabla permite:

- registrar usuarios,
- autenticar sesiones,
- controlar permisos,
- y diferenciar administradores de usuarios normales.

Las contraseñas quedan cifradas con password_hash(), por motivos de seguridad

Estructura tabla camisetas

#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterminado	Comentarios	Extra	Acción
☐ 1	id	int(11)			No	Ninguna		AUTO_INCREMENT	Cambiar Eliminar Más
☐ 2	titulo	varchar(150)	utf8mb4_general_ci		No	Ninguna			Cambiar Eliminar Más
☐ 3	descripcion	text	utf8mb4_general_ci		No	Ninguna			Cambiar Eliminar Más
☐ 4	precio	double			No	Ninguna			Cambiar Eliminar Más
☐ 5	equipo	varchar(100)	utf8mb4_general_ci		No	Ninguna			Cambiar Eliminar Más
☐ 6	imagen	varchar(255)	utf8mb4_general_ci		No	Ninguna			Cambiar Eliminar Más
☐ 7	categoria	varchar(50)	utf8mb4_general_ci		Sí	NULL			Cambiar Eliminar Más
☐ 8	liga	varchar(100)	utf8mb4_general_ci		Sí	NULL			Cambiar Eliminar Más

Esta tabla muestra productos, aplica filtros, edita camisetas y gestiona el catálogo desde el panel del administrador

Estructura tabla pedidos

#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterminado	Comentarios	Extra	Acción
☐ 1	id	int(11)			No	Ninguna		AUTO_INCREMENT	Cambiar Eliminar Más
☐ 2	usuario_id	int(11)			Sí	NULL			Cambiar Eliminar Más
☐ 3	total	double			Sí	NULL			Cambiar Eliminar Más
☐ 4	fecha	timestamp			No	current_timestamp()			Cambiar Eliminar Más

Esta tabla almacena pedidos realizados, guarda el importe y asocia la compra al usuario

Detalle_pedido

#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterminado	Comentarios	Extra	Acción
☐ 1	id	int(11)			No	Ninguna		AUTO_INCREMENT	Cambiar Eliminar Más
☐ 2	pedido_id	int(11)			Sí	NULL			Cambiar Eliminar Más
☐ 3	camiseta_id	int(11)			Sí	NULL			Cambiar Eliminar Más
☐ 4	nombre	varchar(100)	utf8mb4_general_ci		Sí	NULL			Cambiar Eliminar Más
☐ 5	numero	int(11)			Sí	NULL			Cambiar Eliminar Más
☐ 6	talla	varchar(10)	utf8mb4_general_ci		Sí	NULL			Cambiar Eliminar Más
☐ 7	precio	double			Sí	NULL			Cambiar Eliminar Más

La tabla permite almacenar productos comprados, guardar personalizaciones y relacionar productos con pedidos.

4.3 Modelo de procesos y servicios

En este apartado se describen los principales procesos y servicios implementados en la aplicación TopMakki.

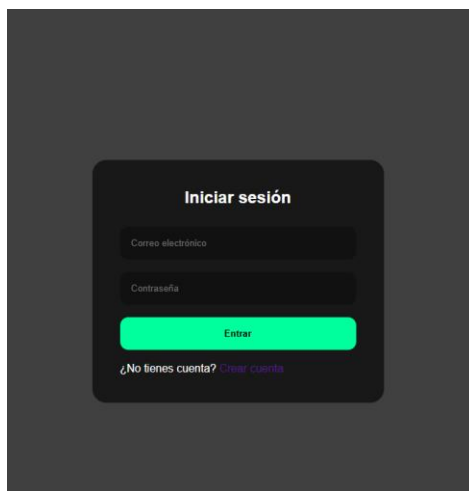
El objetivo de estos procesos es coordinar el funcionamiento interno del sistema y permitir la interacción entre usuarios, productos, carrito y pedidos.

La aplicación se basa en distintos flujos funcionales que conectan las vistas, controladores y base de datos para gestionar correctamente cada operación del ecommerce.

El funcionamiento principal de la aplicación sigue el siguiente proceso:

Usuario → Login → Catálogo → Personalización → Carrito → Checkout → Pedido

El usuario puede navegar por la tienda, seleccionar camisetas, personalizarlas y completar una compra mediante el sistema de carrito y checkout.



Servicio de autenticación

El sistema de autenticación permite controlar el acceso de usuarios y administradores, se encarga de los registros, loggeos, cierres de sesión y de sus validaciones.

El usuario introduce email, contraseña, y el sistema le busca en la base de datos y si no lo encuentra, sale un aviso de credenciales incorrectas, de lo contrario crea una sesión PHP y se redirige acorde al rol del usuario, cabe destacar que el usuario admin es de correo admin@DAW.com y contraseña admin123

El servicio de catálogo gestiona la visualización de los productos disponibles en la tienda, permitiendo al usuario navegar, buscar y acceder a la personalización de camisetas. El

sistema incorpora filtros por liga para facilitar la localización de productos, ofreciendo las siguientes opciones:

- LaLiga
- Premier League
- Serie A
- Bundesliga
- Eredivise
- Selecciones

Estos filtros se aplican mediante consultas SQL dinámicas utilizando PDO. Además, el buscador dinámico desarrollado en JavaScript permite localizar camisetas en tiempo real comparando el texto introducido con los nombres de los equipos y ocultando aquellos productos que no coinciden.

El servicio de personalización permite configurar nombre, número y talla antes de añadir la camiseta al carrito. Toda esta información se almacena temporalmente en la sesión y posteriormente se incluye en el pedido final. El carrito, gestionado mediante sesiones PHP, mantiene los productos seleccionados junto con sus personalizaciones, cantidades y precios, permitiendo añadir, visualizar, eliminar artículos y calcular el total automáticamente.

El proceso de checkout simula la fase final de compra, donde el usuario introduce los datos de pago. Tras la validación, el sistema genera el pedido, lo almacena en la base de datos junto con su detalle y vacía el carrito para completar la operación.

El panel de administración, protegido mediante control de roles y validación de sesiones, permite gestionar el catálogo mediante la creación, edición y eliminación de productos. La comunicación entre vistas, controladores y base de datos sigue el flujo MVC, utilizando PDO para garantizar seguridad y eficiencia en las consultas.

El sistema de autenticación controla el acceso tanto de usuarios como de administradores. Gestiona registros, inicios y cierres de sesión, así como las validaciones necesarias. Cuando el usuario introduce su correo y contraseña, el sistema verifica las credenciales en la base de datos; si no coinciden, se muestra un aviso de error, y si son correctas, se crea una sesión PHP y se redirige al usuario según su rol. El administrador del sistema utiliza las credenciales definidas para el acceso al panel de gestión.

Finalmente, el proyecto incorpora validaciones de sesiones, formularios, roles, consultas preparadas y control de accesos para garantizar un funcionamiento seguro y estable.

5. Construcción del proyecto

5.1 Modelo de procesos y servicios

El proyecto se desarrolló en un entorno local utilizando XAMPP, que proporciona Apache, PHP y MySQL. Desde el inicio se organizó la aplicación en diferentes carpetas para separar correctamente las vistas, los controladores, los recursos públicos y los archivos de configuración. La conexión con la base de datos se implementó mediante PDO en el archivo de configuración principal, utilizando una estructura similar a:

```
php
$conexion = new PDO(
    "mysql:host=localhost;dbname=topmakki",
    "root",
    ""
);
```

- Desarrollo del sistema de usuarios

Uno de los primeros módulos implementados fue el sistema de autenticación. Este módulo permite registrar usuarios, iniciar sesión, mantener sesiones activas y controlar los permisos del administrador. El registro almacena el nombre, el email y la contraseña cifrada mediante `password_hash()`, evitando así guardar contraseñas en texto plano. Durante el inicio de sesión, las credenciales se verifican utilizando `password_verify()`, y si son correctas, se crea una sesión PHP que mantiene al usuario autenticado dentro de la aplicación.

Para diferenciar entre usuarios normales y administradores se implementó un sistema de roles basado en sesiones PHP, utilizando la variable `$_SESSION['usuario']`. Los administradores tienen acceso exclusivo al panel de gestión de productos.

- Desarrollo del catálogo de productos

El catálogo constituye la parte central de la aplicación y muestra todas las camisetas disponibles, incluyendo imagen, nombre, liga, descripción, precio y un botón de compra. Para facilitar la navegación se desarrolló un sistema de filtrado por ligas mediante consultas SQL dinámicas con PDO, por ejemplo:

```
sql
SELECT * FROM camisetas WHERE liga=?
```

Además, se añadió un buscador dinámico en JavaScript que permite localizar camisetas en tiempo real comparando el texto introducido con los nombres de los equipos y ocultando automáticamente las tarjetas que no coinciden.

1. Desarrollo del sistema de personalización

Antes de añadir una camiseta al carrito, el usuario puede personalizar el nombre del dorsal, el número y la talla. Esta información se almacena temporalmente en la sesión y posteriormente se guarda en los pedidos realizados.

2. Desarrollo del carrito de compra

El carrito se implementó utilizando sesiones PHP, almacenando los productos dentro de `$_SESSION['carrito']`. Cada producto guarda la camiseta seleccionada, su precio, la talla, el dorsal, la imagen y la personalización aplicada. Entre las funcionalidades desarrolladas se incluyen la posibilidad de añadir productos personalizados, visualizar el contenido del carrito con todos sus detalles, eliminar artículos individualmente y calcular automáticamente el total acumulado.

3. Desarrollo del checkout

El checkout simula el proceso final de pago de un comercio electrónico. El usuario introduce el nombre del titular, el número de tarjeta y el CVV. Tras confirmar el pago, el sistema genera un pedido, almacena los productos comprados en la base de datos y vacía el carrito para completar la operación.

4. Desarrollo del panel administrador

El panel administrador permite gestionar el catálogo completo de camisetas y solo es accesible para usuarios con rol de administrador. En esta sección se implementaron las operaciones CRUD necesarias: crear nuevos productos, editar los existentes, eliminarlos de la base de datos y mostrarlos en una lista general.

5. Desarrollo frontend

El diseño visual de la aplicación se desarrolló utilizando HTML5, CSS3 y JavaScript, con el objetivo de crear una interfaz moderna, sencilla y fácil de utilizar. La aplicación se adaptó a distintos tamaños de pantalla mediante Flexbox, CSS Grid y Media Queries, garantizando un funcionamiento adecuado tanto en ordenadores como en dispositivos móviles. El diseño visual se basó en colores oscuros, botones modernos, tarjetas de productos, efectos hover y formularios estilizados, buscando simular la apariencia de una tienda online real.

6. Seguridad implementada

Durante el desarrollo se aplicaron diversas medidas de seguridad, como consultas preparadas con PDO, cifrado de contraseñas, validación de sesiones, control de roles y protección del panel administrador. Estas medidas ayudan a prevenir ataques como SQL Injection, accesos no autorizados y manipulaciones incorrectas de datos.

7. Organización del código

El código se organizó separando claramente las vistas, los controladores, los archivos CSS, las imágenes, los scripts JavaScript y la configuración general. Esta estructura facilita el mantenimiento, la lectura del código y futuras ampliaciones del proyecto.

8.Problemas encontrados durante el desarrollo

Durante la construcción del proyecto surgieron diversos problemas técnicos, como errores en las rutas de imágenes, dificultades con las sesiones, validación de roles, problemas de diseño responsive y gestión del carrito. La mayoría de estos inconvenientes se resolvieron mediante depuración del código, reorganización de archivos y pruebas continuas.

9.Resultado final

El resultado final es una aplicación web completamente funcional que permite gestionar usuarios, visualizar productos, realizar compras, administrar camisetas y simular el funcionamiento de un ecommerce real. El sistema integra correctamente frontend, backend y base de datos en una única aplicación dinámica, segura y responsive.

5.2 Modelo de procesos y servicios

Una vez desarrolladas las principales funcionalidades de la aplicación, se llevó a cabo un proceso de integración y pruebas con el objetivo de verificar el correcto funcionamiento del sistema. Estas pruebas permitieron detectar errores, mejorar la estabilidad general y asegurar la comunicación adecuada entre el frontend, el backend y la base de datos.

La integración del sistema se realizó de forma progresiva, conectando cada módulo desarrollado con el resto de componentes de la aplicación. Entre los elementos integrados se encuentran el sistema de usuarios, el catálogo de productos, el carrito de compra, el panel administrador y el sistema de pedidos. Todo el proceso se apoyó en PHP, sesiones, MySQL y consultas PDO para garantizar una comunicación estable y segura.

La integración entre frontend y backend se realizó mediante formularios y peticiones HTTP. El flujo principal de funcionamiento siguió la estructura Vista → Formulario → Controller → Base de datos → Vista. Los controladores se encargaron de procesar la información enviada por el usuario y actualizar tanto la base de datos como las sesiones necesarias para mantener el estado de la aplicación.

La conexión con la base de datos se implementó mediante PDO, integrando correctamente todas las operaciones esenciales del sistema, como la inserción de usuarios, el inicio de sesión, las consultas de productos, la gestión del carrito, la creación de pedidos y la administración de camisetas. Para mejorar la seguridad se utilizaron consultas preparadas, evitando errores SQL y posibles inyecciones.

- Pruebas funcionales realizadas

Se realizaron pruebas funcionales sobre cada uno de los módulos principales. En el sistema de login se comprobó el acceso con usuarios válidos, el rechazo de credenciales incorrectas, el mantenimiento de sesiones y la validación de roles, obteniendo como resultado un sistema

Mohamed El Makki Belahsane

de autenticación estable tanto para usuarios como para administradores. En el registro se verificó la creación de nuevos usuarios, el almacenamiento correcto en la base de datos y el cifrado de contraseñas, confirmando que el proceso funciona correctamente.

En el catálogo se probaron la carga de productos, la visualización de imágenes, los filtros por liga y el buscador dinámico, comprobando que responde de forma fluida y sin errores. El carrito fue probado añadiendo y eliminando productos, verificando el cálculo automático del total y el almacenamiento mediante sesiones, con resultados correctos. El checkout también fue validado comprobando la simulación del pago, la generación de pedidos y el vaciado automático del carrito, confirmando que el flujo de compra funciona adecuadamente.

El panel administrador fue sometido a pruebas de creación, edición, eliminación y listado de camisetas, además de la verificación del acceso restringido únicamente a administradores. Todas las operaciones CRUD funcionaron correctamente. También se realizaron pruebas responsive en ordenador, tablet y móvil, comprobando la correcta adaptación de tarjetas, menús, formularios y el carrito mediante las herramientas de desarrollo del navegador.

- Problemas detectados

Durante las pruebas se detectaron diversos errores, como rutas incorrectas de imágenes, problemas con sesiones, fallos en el carrito, validación incorrecta de arrays y errores responsive en algunas tarjetas. Estos problemas se solucionaron mediante reorganización de rutas, validaciones adicionales en PHP, ajustes en CSS y pruebas continuas hasta asegurar un funcionamiento estable.

- Validaciones implementadas

Para mejorar la estabilidad del sistema se añadieron validaciones de formularios, comprobación de sesiones activas, control de roles y validación de datos recibidos. Además, se utilizaron consultas preparadas PDO para evitar SQL Injection, errores SQL y accesos indebidos.

- Herramientas utilizadas para las pruebas

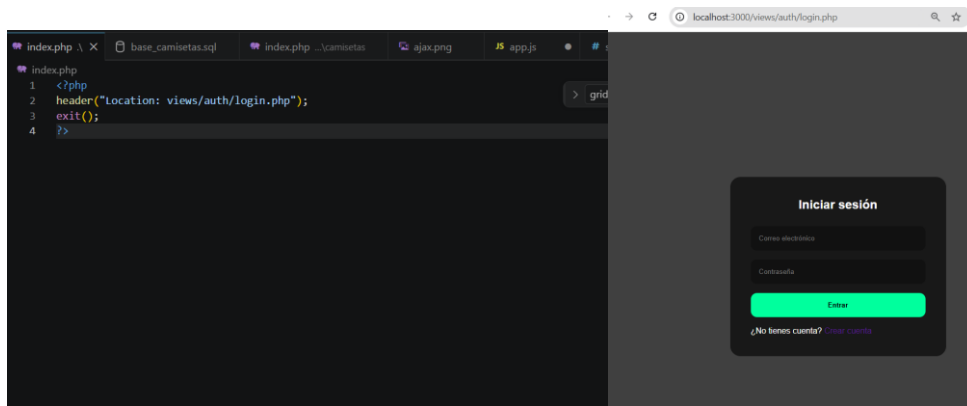
Las pruebas se realizaron utilizando Google Chrome, Mozilla Firefox, XAMPP, phpMyAdmin y Visual Studio Code. También se emplearon las herramientas de desarrollador del navegador para depurar errores JavaScript, estilos CSS y peticiones HTTP.

- Resultado final de las pruebas

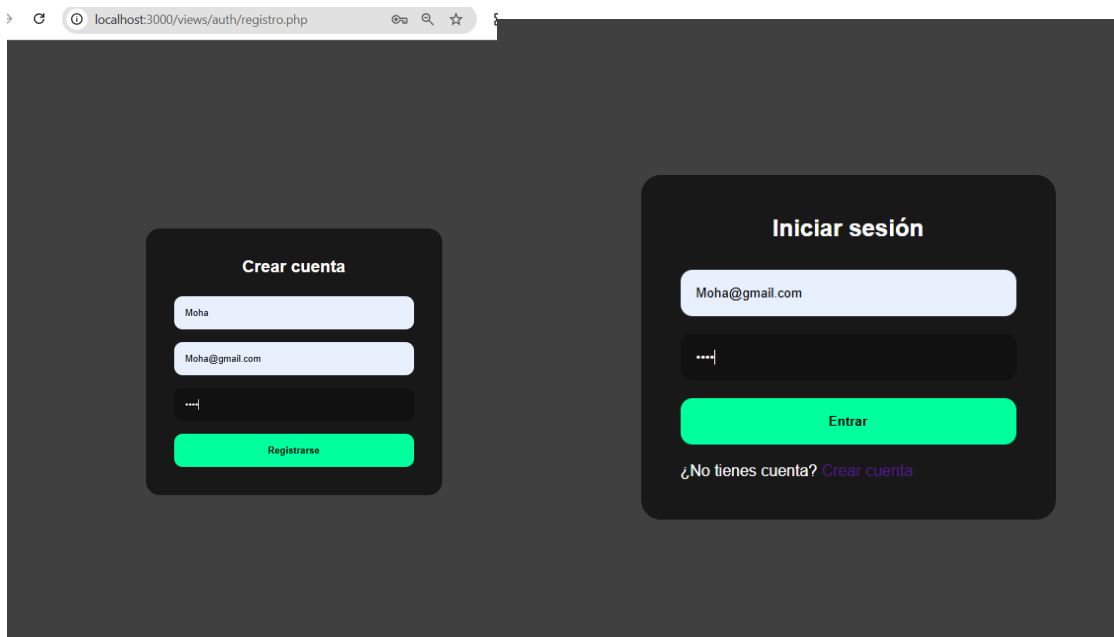
Tras completar el proceso de integración y pruebas, la aplicación alcanzó un funcionamiento estable y coherente entre todos sus módulos. El sistema final permite gestionar usuarios, administrar productos, realizar compras, almacenar pedidos y controlar sesiones de forma correcta. Las pruebas realizadas contribuyeron a mejorar la calidad general de la aplicación y a reducir errores durante su uso.

5.3 Manual de usuario

Paso 1: nos situamos en el index.php que hay en el directorio raíz y lo ejecutamos, nos llevara al login obligatoriamente para iniciar sesión, de no tener cuenta crearemos una.



Paso 2: como no tenemos cuenta previa, iremos a crear una, donde nos pedirá nombre, correo y contraseña, una vez registrados volvemos al login e iniciamos sesión, esto es gracias a los controladores `registroController` y `loginController`, que actúan respectivamente sobre `registro.php` y `login.php`, donde tienen las consultas que se ejecutan para que funcionen, depende del loggeo es usuario admin o comprador:



```

1 <?php
2 require_once('../config/conexion.php');
3
4 if (isset($_POST['registro'])) {
5
6     $nombre = $_POST['nombre'];
7     $email = $_POST['email'];
8     $password = password_hash($_POST['password'], PASSWORD_DEFAULT);
9
10    $stmt = $conexion->prepare(
11        "INSERT INTO usuarios (nombre, email, password)
12        VALUES (?, ?, ?)"
13    );
14
15    $stmt->execute([
16        $nombre,
17        $email,
18        $password
19    ]);
20
21    header("Location: ../views/auth/login.php");
22 }
23 ?>
24
<?php
session_start();
require_once('../config/conexion.php');

if (isset($_POST['login'])) {

    $email = $_POST['email'];
    $password = $_POST['password'];

    $stmt = $conexion->prepare("SELECT * FROM usuarios WHERE email=?");
    $stmt->execute([$email]);

    $usuario = $stmt->fetch();

    if ($usuario && password_verify($password, $usuario['password'])) {

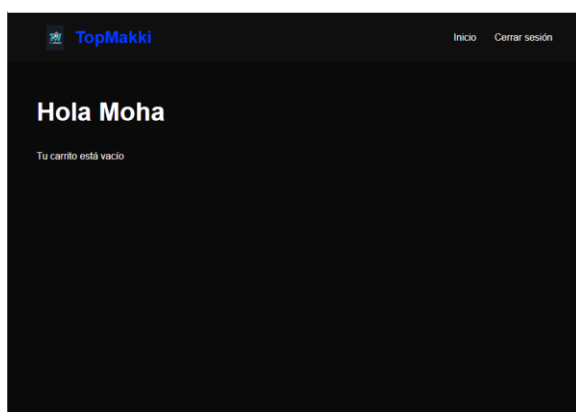
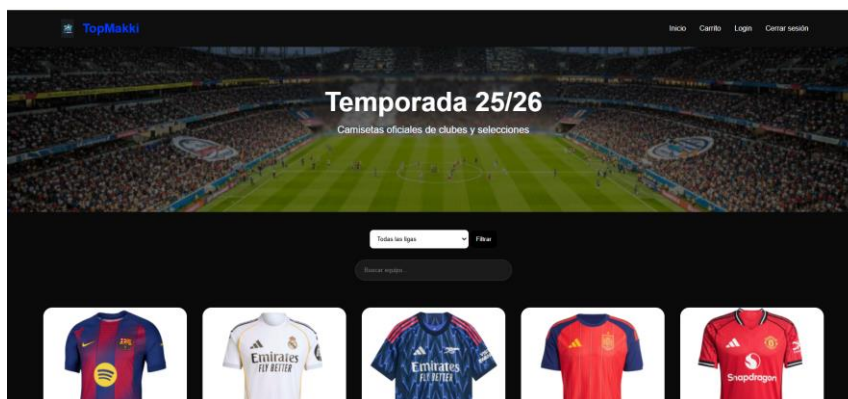
        $_SESSION['id'] = $usuario['id'];
        $_SESSION['nombre'] = $usuario['nombre'];
        $_SESSION['rol'] = $usuario['rol'];
        $_SESSION['usuario'] = $usuario;

        if ($usuario['rol'] == 'admin') {
            header("Location: ../views/admin/dashboard.php");
        } else {
            header("Location: ../views/camisetas/index.php");
        }

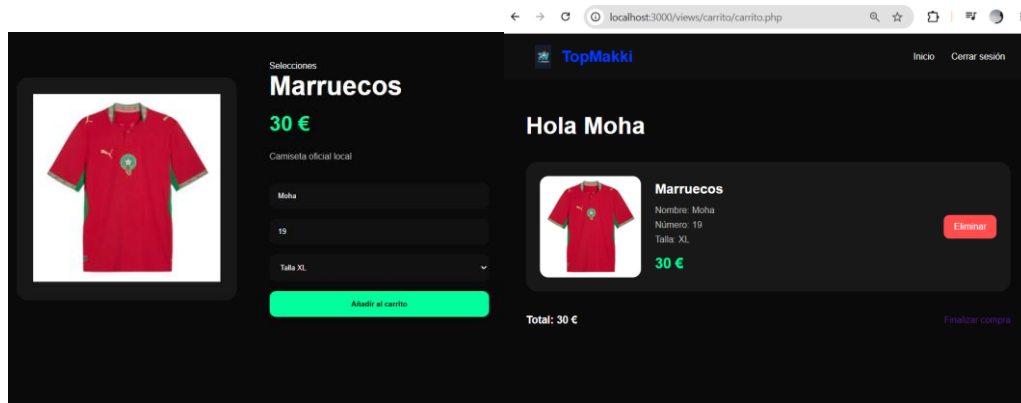
    } else {
        echo "Credenciales incorrectas";
    }
}
?>

```

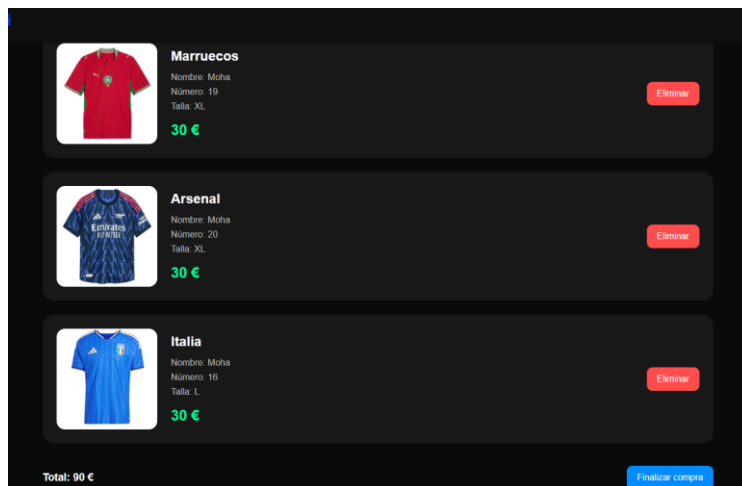
Paso 3: ya en la pagina como rol de usuario, se nos abre la tienda la cual podemos escoger en una amplia gama de camisetas, todas con un precio uniforme de 30€, capaz también de buscar con un buscador o filtrar por ligas, si entramos al carrito veremos que esta vacío



Paso 4: como está vacío, volvemos a la tienda para comprar algo y testearlo, pinchamos en una camiseta y ya nos sale personalizarla, una vez personalizada la añadimos al carrito y nos lleva ahí inmediatamente:



Nos sale el total y con la opción de eliminar, añadimos 2 más para verlo mejor



Paso 5: vamos a comprar, así que le damos a finalizar compra:

Finalizar compra

Total: 90 €

[Pagar ahora](#)

Finalizar compra

Total: 90 €

[Pagar ahora](#)

Pedido realizado

Tu compra se ha procesado correctamente.

[Volver a tienda](#)

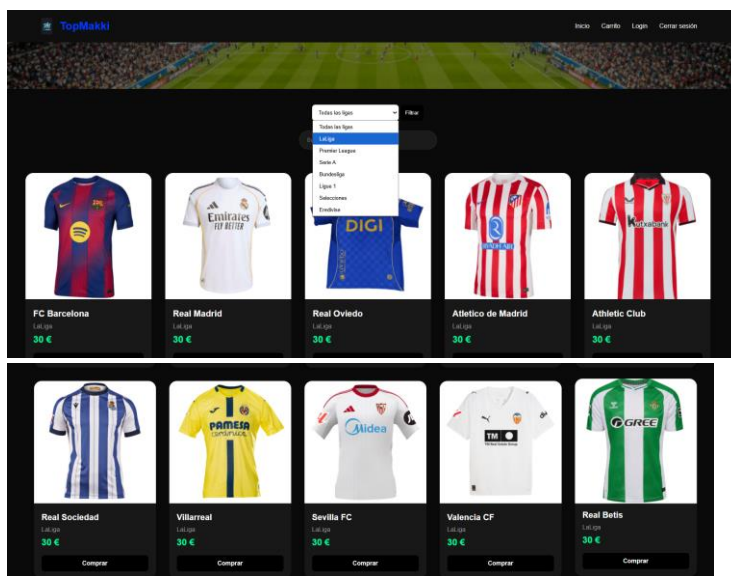
Paso 6: El pedido se guarda en la base de datos en la tabla pedidos, y en detalle_pedidos tambien se graba

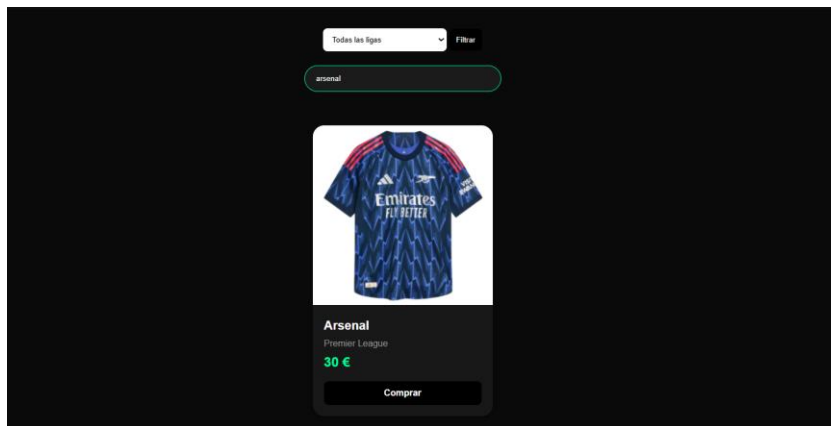
←T→		id	usuario_id	total	fecha
☐	 Editar	 Copiar	 Borrar	1	6 90 2026-05-15 12:03:28
☐	 Editar	 Copiar	 Borrar	2	6 60 2026-05-15 15:41:09
☐	 Editar	 Copiar	 Borrar	3	1 60 2026-05-15 17:17:37
☐	 Editar	 Copiar	 Borrar	4	7 60 2026-05-15 23:48:32
☐	 Editar	 Copiar	 Borrar	5	4 90 2026-05-16 10:26:50

Es el ultimo

←T→				id	pedido_id	camiseta_id	nombre	numero	talla	precio
☐				1	1	NULL	Pablo	15	XXL	30
☐				2	1	NULL	luis	14	S	30
☐				3	1	NULL	Dani	12	XXL	30
☐				4	2	NULL	luis	11	S	30
☐				5	2	NULL	ana	14	XL	30
☐				6	3	NULL	omaimé	7	L	30
☐				7	3	NULL	omaime	7	L	30
☐				8	4	NULL	Abde	10	M	30
☐				9	4	NULL	hakimi	2	M	30
☐				10	5	NULL	Moha	19	XL	30
☐				11	5	NULL	Moha	20	XL	30
☐				12	5	NULL	Moha	16	L	30

Paso 7: Volviendo a la página principal, podemos ver como filtrar por equipos en el buscador o por ligas





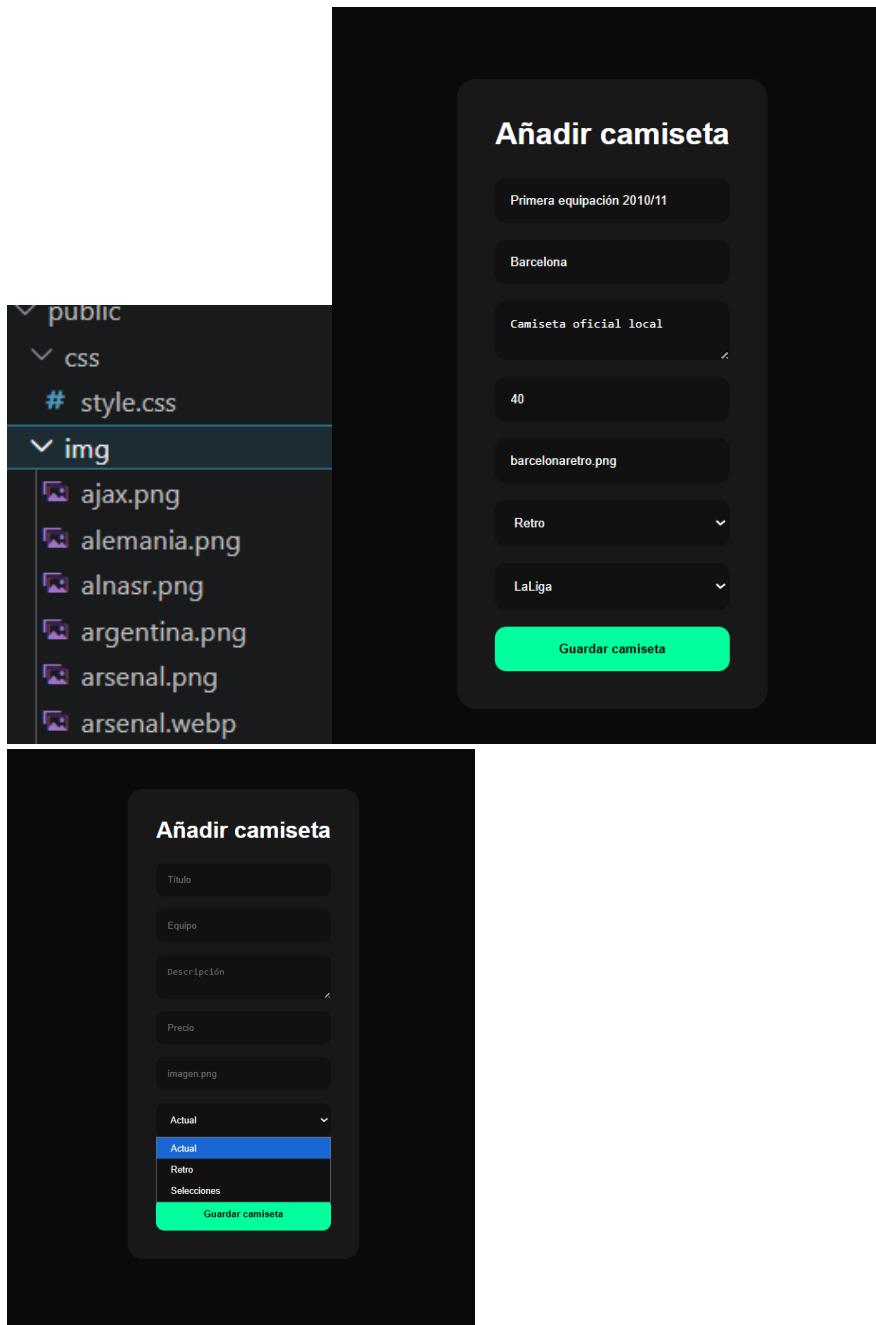
Paso 8: Cerramos sesión y ahora entramos con el usuario administrador:
admin@DAW.com y admin123, se accede al panel donde se controlan todas las camisetas

Panel administrador

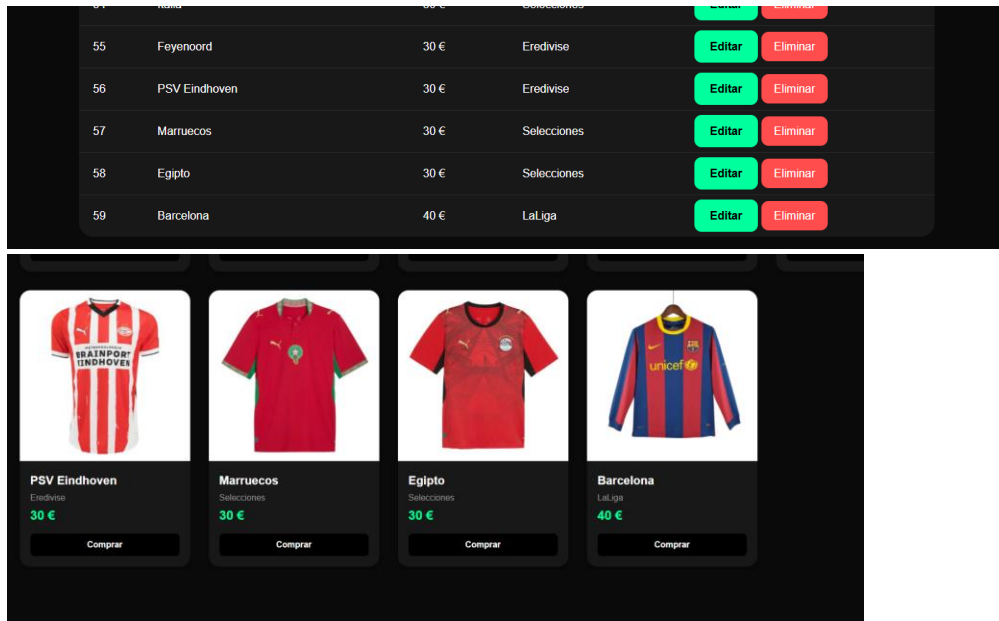
Añadir camiseta

ID	Equipo	Precio	Liga	Acciones	
1	FC Barcelona	30 €	LaLiga	Editar	Eliminar
2	Real Madrid	30 €	LaLiga	Editar	Eliminar
3	Arsenal	30 €	Premier League	Editar	Eliminar
4	España	30 €	Selecciones	Editar	Eliminar
5	Manchester United	30 €	Premier League	Editar	Eliminar
6	Ajax	30 €	Eredivise	Editar	Eliminar
7	Real Oviedo	30 €	LaLiga	Editar	Eliminar
8	Milan	30 €	Serie A	Editar	Eliminar
9	Atletico de Madrid	30 €	LaLiga	Editar	Eliminar
10	Athletic Club	30 €	LaLiga	Editar	Eliminar
11	Bayern Munich	30 €	Bundesliga	Editar	Eliminar

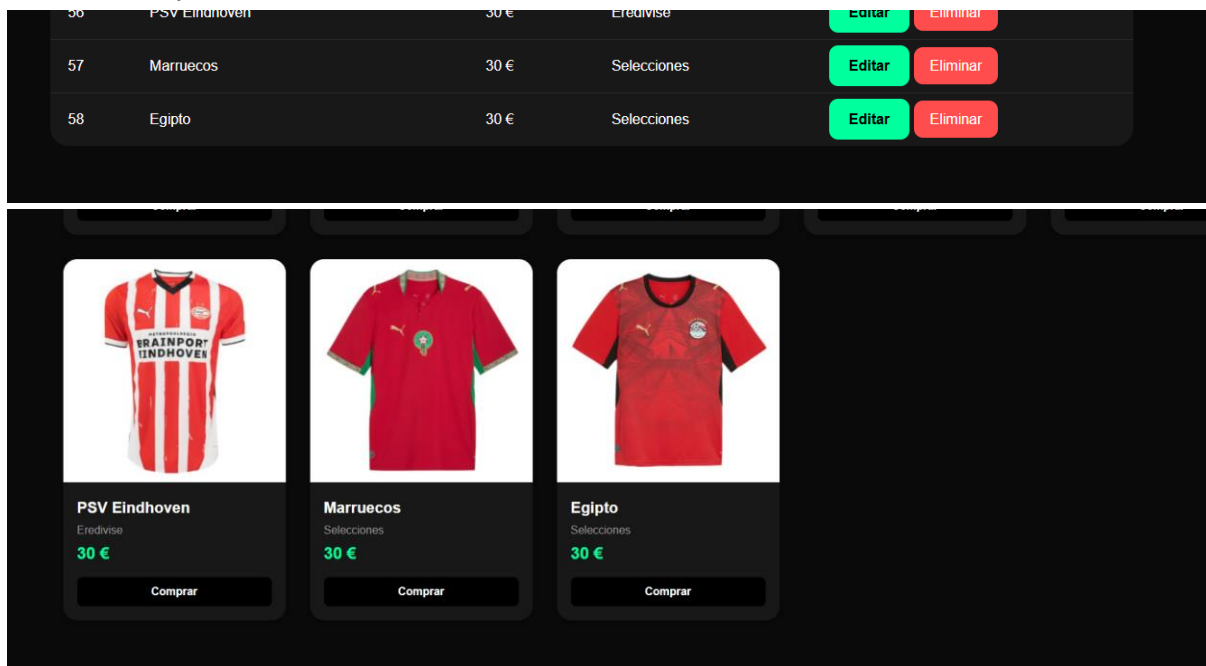
Paso 9: Como prueba, vamos a probar a crear una y luego borrarla, contando con la imagen en la carpeta public/img



Paso 10: vemos como se ha creado la camiseta, entramos a la tienda para verla, a posteriori la eliminamos y vemos que no esta



Y cuando lo pulsamos, se va:



Paso 11: ahora probaremos a editar alguna camiseta, para ello, buscaremos una al azar y le subiremos el precio, por ejemplo la del Como, la subiremos a 50, y le pondremos de liga la Serie A al tener la liga mal:

En efecto, funciona:

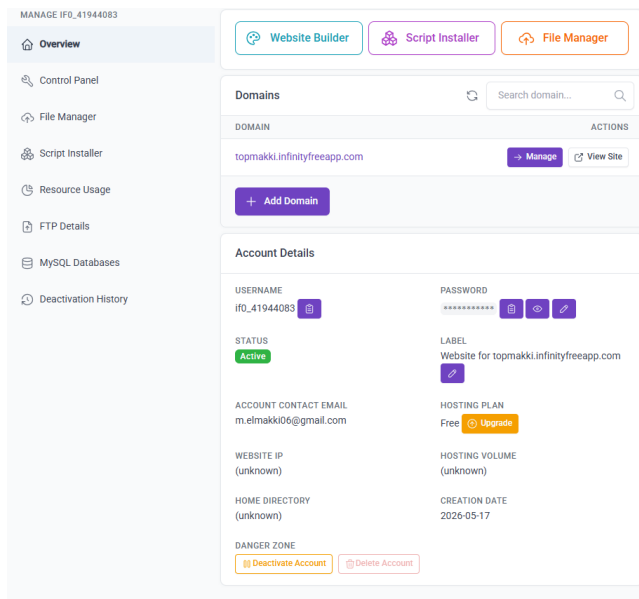
```
<!-- NAVBAR -->
<nav class="navbar">
  <a href="#" class="logo">
    
    <span>TopMakki</span>
  </a>
  <ul class="menu">
    <li><a href="#">Inicio</a></li>
    <li><a href="../carrito/carrito.php">Carrito</a></li>
    <li><a href="/logout.php">Cerrar sesión</a></li>
  </ul>
</nav>
```

```
logout.php X index.php \ base_camisetas.sql index
logout.php
1  <?php
2
3  session_start();
4
5  session_destroy();
6
7  header("Location: views/auth/login.php");
8
9  exit();
10 ?>
```

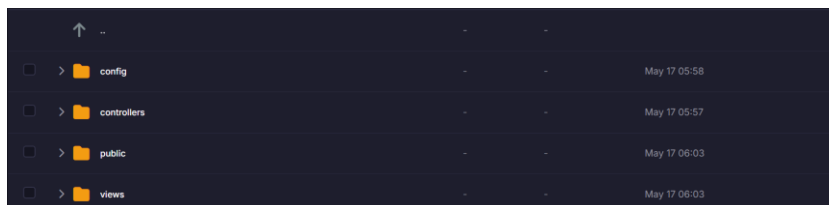
Como recomendación, se aconseja usar navegadores como Google Chrome o Edge, mantener Apache y SQL activos en XAMPP y no modificar manualmente la base de datos si se está utilizando la aplicación, esta diseñada para el uso de administradores y así también más enfocado para los usuarios.

5.4 Manual de instalación y despliegue

Paso 1: voy a hacer el despliegue en infinity free, así que para ello primero creamos la cuenta y el dominio y subimos la aplicación



Paso 2: subimos la aplicación en httdocs, cuando se nos crea



Paso 3: la base de datos se va a crear, para ello en el panel vamos a MySQL Databases y la creamos, una vez creada vamos a phpMyAdmin e importamos la base

MySQL Connection Details

Mysql USERNAME
if0_41944083

Mysql PASSWORD

Mysql DATABASE NAME
if0_41944083_XXX
See below for available database names

Mysql HOSTNAME
sql102.infinityfree.com

Mysql PORT (OPTIONAL)
3306

List of MySQL Databases

DATABASE NAME	ACTIONS
if0_41944083_base_camisetas	phpMyAdmin Delete

+ Create Database

How to use MySQL

MySQL is a relational database system. If you want your website to have any kind of interactive functionality, like social functions or shopping features, or just want a way to manage your website's content easily without code, almost all websites use a MySQL database to store the data.

Are you having trouble establishing a database connection? Please see the article [Common MySQL Connection Errors](#) for more information.

Servidor: sql102.infinityfree.com »
Base de datos: if0_41944083_base_camisetas

Estructura
SQL
Buscar
Generar una consulta
Exportar
Importar

Importación ejecutada exitosamente, 27 consultas ejecutadas. (base_camisetas.sql)

MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0.0003 segundos.)

-- phpMyAdmin SQL Dump -- version 5.2.1 -- https://www.phpmyadmin.net/ -- -- Servidor: 127.0.0.1 -- Tiempo de generación: 17-05-2026 a las 13:14:29 -- Versión del servidor: 10.4.32-MariaDB -- Versión de PHP: 8.0.30 SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO"

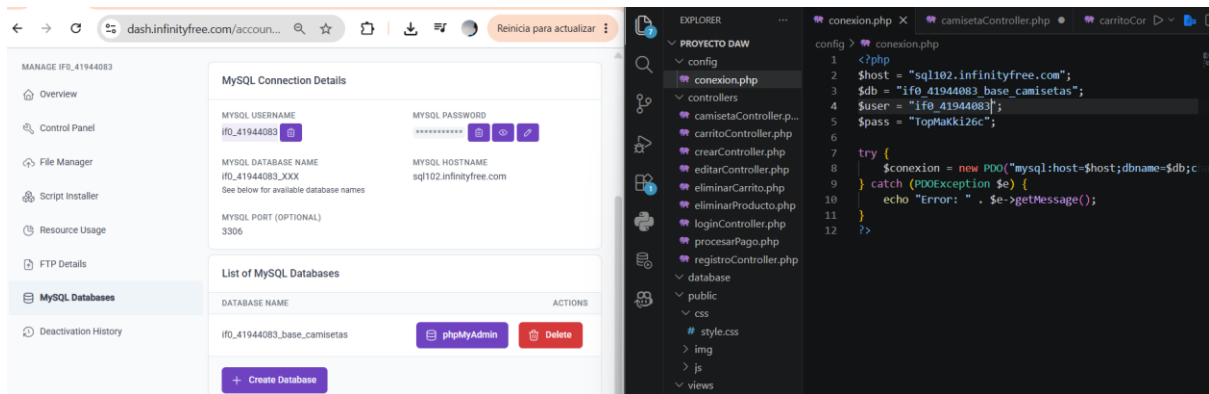
[Editar en línea] [Editar] [Crear código PHP]

MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0.0002 segundos.)

START TRANSACTION

[Editar en línea] [Editar] [Crear código PHP]

Paso 4: una vez subido actualizamos la base con los parámetros para conectarla:

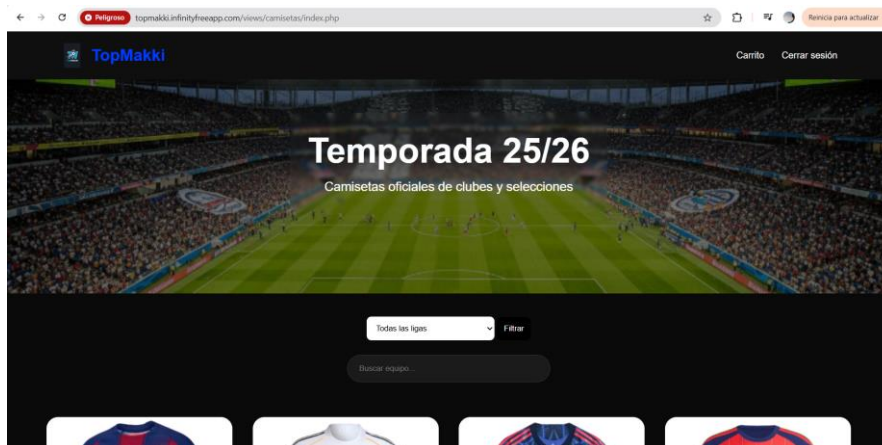


Lo guardamos en el que esta subido y luego comprobamos que funciona

Paso 5: una vez subido actualizamos la base con los parámetros para conectarla:

Comprobamos que la base se ha creado correctamente

Paso 6: iniciamos sesión y comprobamos que funciona, la aplicación está desplegada aquí: topmakki.infinityfreeapp.com



6.Evaluación final del proyecto

6.1 Evaluación del diseño y resultado final

Al comienzo del proyecto se planteó desarrollar una aplicación web sencilla orientada a la venta de camisetas de fútbol personalizadas.

La idea inicial consistía en crear:

- un catálogo básico de productos,
- sistema de usuarios,
- carrito de compra,
- y panel administrador.

En un principio el diseño pensado era bastante más simple, con una interfaz básica y pocas funcionalidades dinámicas. Sin embargo, a medida que avanzó el desarrollo, el proyecto fue evolucionando y mejorando tanto a nivel visual como funcional.

Finalmente, la aplicación terminó incluyendo:

- autenticación de usuarios,
- control de roles,
- catálogo dinámico,
- filtros y buscador,
- personalización de camisetas,
- carrito funcional,
- checkout,
- y panel de administración completo.

- **Evolución del diseño visual**

Inicialmente el diseño iba a desarrollarse utilizando una estructura muy simple basada únicamente en HTML y CSS básicos.

Durante el desarrollo se decidió mejorar considerablemente el aspecto visual:

- utilizando tarjetas modernas,
- efectos hover,
- diseño oscuro,
- formularios estilizados,
- y diseño responsive.

Aunque se valoró utilizar Bootstrap, finalmente se optó por desarrollar el diseño manualmente mediante CSS propio para tener un mayor control visual sobre la aplicación.

El resultado final ofrece una apariencia más cercana a una tienda online real y una experiencia de usuario más moderna.

Evolución técnica

En cuanto a la estructura del proyecto, inicialmente se planteó una organización muy básica.

Conforme avanzó el desarrollo se reorganizó el código separando:

- vistas,
- controladores,
- recursos públicos,
- y configuración.

También se implementaron mejoras importantes como:

- consultas preparadas PDO,
- cifrado de contraseñas,
- validación de sesiones,
- y control de acceso para administradores.

Resultado final obtenido

El resultado final es una aplicación web funcional capaz de:

- gestionar usuarios,
- mostrar productos,
- realizar compras,
- personalizar camisetas,
- administrar el catálogo,
- y almacenar pedidos.

La aplicación consigue integrar correctamente:

- frontend,
- backend,
- y base de datos.

Además, el sistema funciona de forma responsive en diferentes dispositivos.

Aspectos positivos del resultado

Entre los puntos más positivos del proyecto destacan:

TopMakki

- organización del código,
- diseño moderno,
- funcionalidad completa del ecommerce,
- panel administrador funcional,
- y buena integración entre tecnologías.

También destaca el hecho de haber desarrollado prácticamente todo el frontend manualmente utilizando CSS propio.

Aspectos mejorables

Aunque el resultado es satisfactorio, todavía existen posibles mejoras:

- implementar pagos reales,
- mejorar validaciones,
- añadir subida de imágenes,
- historial de pedidos,
- y un sistema MVC más avanzado.

También podrían optimizarse algunos apartados responsive y mejorar la seguridad general del sistema.

6.2 Conclusiones y lecciones aprendidas

La realización del proyecto TopMakki ha permitido aplicar de forma práctica muchos de los conocimientos aprendidos durante el ciclo formativo DAW.

A lo largo del desarrollo se trabajó con tecnologías tanto de frontend como de backend, integrando todos los elementos necesarios para crear una aplicación web dinámica y funcional.

Conocimientos adquiridos

El proyecto permitió reforzar conocimientos sobre:

- PHP,
- MySQL,
- HTML5,
- CSS3,
- JavaScript,
- sesiones,
- consultas PDO,
- y diseño responsive.

Además, ayudó a comprender mejor:

- la estructura de aplicaciones web,
- la comunicación entre cliente y servidor,
- y el funcionamiento interno de un ecommerce.

Aprendizaje práctico

Uno de los aspectos más importantes fue aprender a resolver problemas reales durante el desarrollo.

Muchos errores aparecieron relacionados con:

- rutas,
- sesiones,
- consultas SQL,
- carrito,
- y responsive.

La resolución de estos problemas permitió mejorar la capacidad de:

- depuración,
- organización del código,
- y búsqueda de soluciones.

Importancia de la organización

Durante el proyecto también se aprendió la importancia de:

- estructurar correctamente carpetas,
- separar responsabilidades,
- y mantener un código organizado.

Esto facilitó considerablemente el mantenimiento y desarrollo progresivo de la aplicación.

7. Bibliografía y recursos

Durante el desarrollo del proyecto se utilizaron diferentes recursos y fuentes de información para resolver dudas técnicas y apoyar el desarrollo de la aplicación.

Documentación oficial

- PHP Documentation

- MySQL Documentation
- MDN Web Docs
- W3Schools
- Stack Overflow

Tecnologías utilizadas

- PHP
- MySQL
- HTML5
- CSS3
- JavaScript
- PDO
- XAMPP
- phpMyAdmin
- Visual Studio Code

Recursos gráficos

Las imágenes utilizadas para las camisetas y fondos fueron obtenidas de:

- Google Imágenes
- recursos libres de internet
- imágenes propias utilizadas únicamente con fines educativos

Herramientas de desarrollo

- XAMPP
- Visual Studio Code
- Google Chrome DevTools
- PhpMyAdmin

8.Anexos

- [Diseño responsive](#)

